

Hướng dẫn cài đặt vLLM & 3 Model AI trên cụm 2 máy ASUS Ascent GX10

Sản phẩm: ASUS Ascent GX10 · GPU NVIDIA Blackwell GB10 · ARM v9.2-A · 128 GB LPDDR5x Unified Memory/node **Hệ điều hành:** NVIDIA DGX™ Base OS (Ubuntu Linux) — cài sẵn, OS duy nhất được hỗ trợ chính thức **Mạng nội bộ:** Node 1 (192.168.100.10/24) ↔ Node 2 (192.168.100.11/24) qua NVIDIA ConnectX-7 200G **Models:** MedGemma 27B-IT · MedGemma 1.5 4B-IT · Llama 4 Scout 17B (chạy song song) **Tài liệu tham khảo:** [ASUS Ascent GX10](#) · [ASUS FAQ GX10](#) · [NVIDIA DGX Spark Playbooks](#)

Thông số kỹ thuật ASUS Ascent GX10

Nguồn: [ASUS Ascent GX10 Tech Specs](#)

Thành phần	Thông số
Tên sản phẩm	ASUS Ascent GX10
CPU	ARM v9.2-A (GB10 Superchip)
GPU	NVIDIA Blackwell GPU (GB10, integrated)
Memory	128 GB LPDDR5x — unified system memory (CPU + GPU dùng chung)
Storage	1 TB M.2 NVMe PCIe 4.0 SSD
Mạng	1× NVIDIA ConnectX-7 SmartNIC (200G QSFP) · 1× 10G LAN · Wi-Fi 7 · Bluetooth 5.4
Cổng I/O	3× USB 3.2 Gen 2×2 Type-C · 1× USB-C PD 180 W · 1× HDMI 2.1 · 1× Kensington Lock
Nguồn	240 W Adapter
Kích thước	150 × 150 × 51 mm · 1,48 kg
OS	Ubuntu Linux (NVIDIA DGX™ Base OS) — OS duy nhất được kiểm tra và hỗ trợ chính thức
Hiệu năng AI	1 petaFLOP (FP4)
Driver	Dòng 580/590-open — dành riêng cho Blackwell GB10
CUDA	Phiên bản 13.x — GB10 (sm_121) yêu cầu CUDA 13 trở lên
nvidia-smi	Báo "Memory-Usage: Not Supported" — hành vi bình thường của unified memory
vLLM	Yêu cầu wheel CUDA 13 / aarch64 — không dùng <code>pip install</code> thông thường
Swap	Phải tắt trước khi vận hành — OOM trên unified memory có thể làm treo toàn bộ hệ thống
Cập nhật hệ thống	Thực hiện qua DGX Dashboard (<code>http://localhost:11000</code>) — không dùng <code>apt upgrade</code>

Thành phần	Thông số
Cluster	Hỗ trợ kết nối tối đa 2 node qua QSFP 200G trực tiếp, hoặc nhiều hơn qua 200G switch

Mục lục

- [I. Phân tích tài nguyên](#)
- [II. Cấu hình mạng 200G — RoCE + MTU](#)
- [III. Driver & CUDA](#)
- [IV. Cài vLLM](#)
- [V. Chạy các model](#)
- [VI. Kiểm tra & giám sát](#)
- [VII. Liên hệ hỗ trợ](#)

I. Phân tích tài nguyên

1. Bộ nhớ thực tế vận hành (FP8)

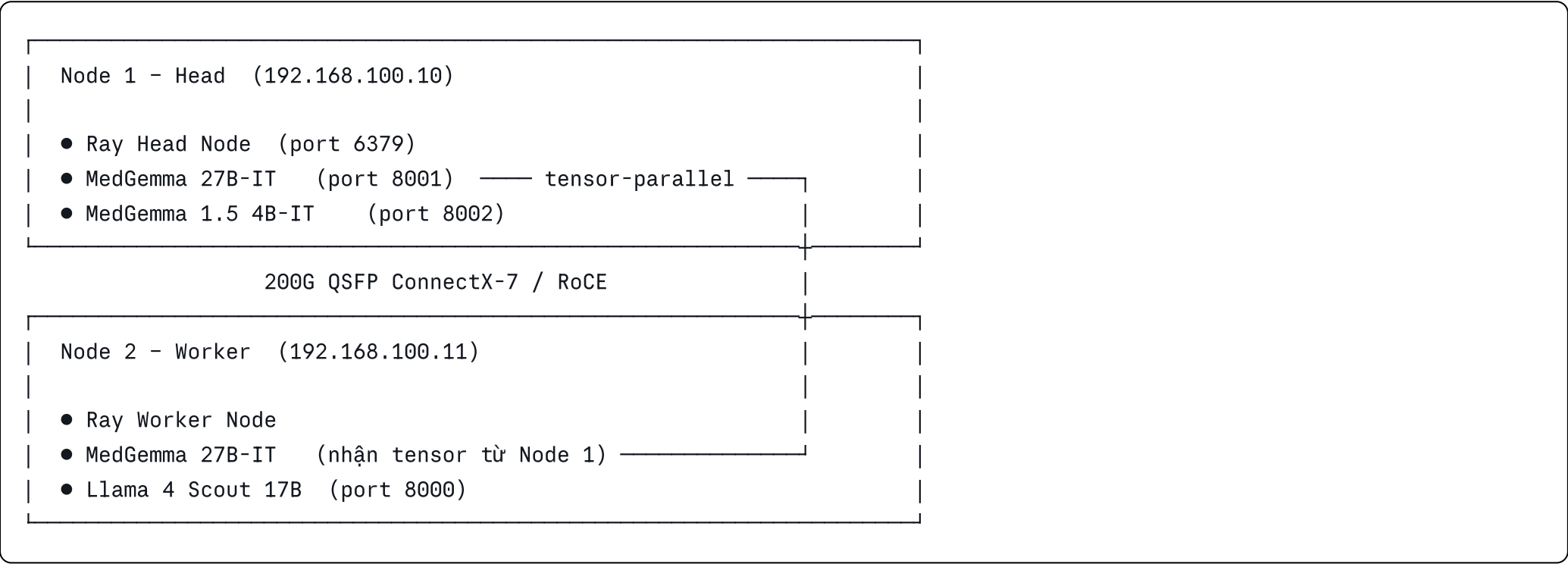
Model	Kiến trúc	Params (active / total)	Bộ nhớ lý thuyết (FP8)	Bộ nhớ thực tế	Mức dùng GPU	Cấu hình Ray	Node chạy
MedGemma 27B-IT	Gemma 3 27B (dense)	27B / 27B	~27 GB	~32 GB / node	0.36	pp=2 (2 nodes)	Node 1 + Node 2
MedGemma 1.5 4B-IT	PaliGemma 2 (SigLIP 400M + Gemma 3B)	~3.4B / ~3.4B	~3.5 GB	~10 GB	0.1	single node	Node 1

Model	Kiến trúc	Params (active / total)	Bộ nhớ lý thuyết (FP8)	Bộ nhớ thực tế	Mức dùng GPU	Cấu hình Ray	Node chạy
Llama 4 Scout 17B	MoE 16 experts (NVIDIA FP8 pre-quant)	17B / 109B	~54 GB	~70 GB / node	0.5	pp=2 (2 nodes) ¹	Node 1 + Node 2
Tổng			~84.5 GB	~174 GB			2 × 128 GB

¹ Llama 4 Scout không khởi động được trên 1 node đơn — bắt buộc pipeline-parallel 2 nodes.

128 GB trên mỗi máy là bộ nhớ dùng chung giữa CPU và GPU. Khi hệ thống hết bộ nhớ, toàn bộ máy có thể bị treo thay vì chỉ crash một process. Blackwell GB10 có Tensor Core thế hệ 5 với hardware accelerator riêng cho FP8 và FP4, giúp tăng throughput ~2× so với FP16.

2. Kiến trúc triển khai



Port tổng hợp:

- 8000 — Llama 4 Scout 17B (Node 2)
 - 8001 — MedGemma 27B-IT (Node 1, tensor-parallel sang Node 2)
 - 8002 — MedGemma 1.5 4B-IT (Node 1)
 - 6379 — Ray Head (Node 1)
-

3. Kiểm tra hệ thống & cấu hình nền

Thực hiện trên cả 2 máy

3.1 Kiểm tra GPU và ConnectX-7

```
bash

# Kiểm tra GPU GB10
lspci | grep -i nvidia
# Kỳ vọng: 000f:01:00.0 VGA compatible controller: NVIDIA Corporation Device 2e12

# Kiểm tra ConnectX-7 NIC
lspci | grep -i mellanox
# Kỳ vọng: ít nhất 2 dòng Mellanox MT2910 Family [ConnectX-7]

# Kiểm tra driver và phiên bản CUDA
nvidia-smi
# "Memory-Usage: Not Supported" là hành vi bình thường của unified memory

# Kiểm tra phiên bản CUDA
nvcc --version
# Kỳ vọng: CUDA 13.x
```

3.2 Tắt swap

Bắt buộc — Trên unified memory, OOM killer không hoạt động theo cơ chế thông thường; swap sẽ gây treo hệ thống.

```
bash

sudo swapoff -a

# Xác nhận đã tắt (không có output = đã tắt hoàn toàn)
swapon --show

# Tắt vĩnh viễn khi khởi động lại
sudo sed -i '/swap/s/^/#/' /etc/fstab
```

3.3 Cài công cụ cần thiết

```
bash

# Chỉ update danh sách package – KHÔNG dùng apt upgrade
sudo apt update

sudo apt install -y build-essential curl wget git \
    rdma-core infiniband-diags perftest \
    python3-pip
```

II. Cấu hình mạng 200G — RoCE + MTU

Tài liệu tham khảo: [NVIDIA DGX Spark — Spark Stacking](#) · [NVIDIA dgx-spark-playbooks](#)

⚠ **Bước này phải hoàn thành trước tất cả các bước cài đặt phía sau.** Nếu mạng 200G chưa được cấu hình đúng, NCCL và tensor-parallel sẽ không đạt đủ băng thông hoặc không hoạt động.

1. Kiến trúc mạng ConnectX-7

ConnectX-7 trên GX10 sử dụng kiến trúc **twin interface**: mỗi cổng QSFP vật lý chia ra 2 PCIe x4 link, tạo thành 4 interface logic:

```
1 cổng QSFP vật lý
├─ PCIe x4 link 1 → enp1s0f0np0    (Ethernet)
│                   → rocep1s0f0    (RoCE/RDMA)
└─ PCIe x4 link 2 → enp1s0f1np1    (Ethernet)
                   → rocep1s0f1    (RoCE/RDMA)
```

! Mỗi twin cung cấp tối đa ~100G. Phải sử dụng cả 2 twin trong cấu hình NCCL để đạt full 200G.

2. Thiết lập Netplan khuyến nghị

Thực hiện trên **cả 2 máy**:

```
bash

sudo wget -O /etc/netplan/40-cx7.yaml \
  https://github.com/NVIDIA/dgx-spark-playbooks/raw/main/nvidia/connect-two-sparks/assets/cx7-netplan.yaml
sudo chmod 600 /etc/netplan/40-cx7.yaml
sudo netplan apply
```

Tạo thủ công khi không thể kết nối bằng `wget` (thay thế nội dung file `cx7-netplan.yaml`):

```
yaml
```

```
network:
  version: 2
  ethernets:
    enp1s0f0np0:
      link-local: [ ipv4 ]
    enp1s0f1np1:
      link-local: [ ipv4 ]
```

Sau khi tạo file:

```
bash

sudo chmod 600 /etc/netplan/40-cx7.yaml
sudo netplan apply

# Xác nhận interface đã up
ip addr show enp1s0f0np0
ip addr show enp1s0f1np1
```

4. Kiểm tra RDMA / RoCE

```
bash
```



```
# Xem danh sách RDMA devices
ibv_devices

# Kỳ vọng: mlx5_0, mlx5_1 (2 virtual ports của ConnectX-7)

# Kiểm tra trạng thái link
rdma link

# Kỳ vọng: state ACTIVE

# Kiểm tra kernel module
lsmod | grep mlx5

# mlx5_core và mlx5_ib phải có mặt
```

Bandwidth test (RDMA write):

```
bash

# Node 1 – server mode (chạy trước)
ib_write_bw -d mlx5_0 -i 1 -p 12000 -F --report_gbits --run_infinately

# Node 2 – client mode (chạy sau, trở vào IP Node 1)
ib_write_bw -d mlx5_0 -i 1 -p 12000 -F --report_gbits 192.168.100.10

# Kỳ vọng: ~11.7 GB/s per twin
# Tổng 2 twin: ~23.4 GB/s (giới hạn thực tế của PCIe 5.0 x4 + link encoding)
```

5. Thiết lập SSH passwordless (cho Ray)

NVIDIA cung cấp script tự động — chạy trên **Node 1**:

```
bash
```

```
wget https://github.com/NVIDIA/dgx-spark-playbooks/raw/refs/heads/main/nvidia/connect-two-sparks/assets/discover-sparks
chmod +x discover-sparks
./discover-sparks
```

Kiểm tra SSH không cần password:

```
bash

# Từ Node 1 sang Node 2
ssh user@192.168.100.11 hostname
# Kỳ vọng: trả về hostname của Node 2, không hỏi password
```

6. Quy hoạch mở rộng

Subnet /24 cho phép gán thêm node mới chỉ bằng cách thêm IP mà không cần thay đổi cấu hình mạng hiện có:

Node	IP
Node 1 (hiện tại)	192.168.100.10
Node 2 (hiện tại)	192.168.100.11
Node 3 (mở rộng)	192.168.100.12
Node 4+	192.168.100.13...

| Khi mở rộng lên 3 node trở lên, cần bổ sung L2 switch 200G (ví dụ Mikrotik CRS812-DDQ hỗ trợ 8× 200G QSFP).

III. Driver & CUDA

GX10 đã có DGX OS với driver cài sẵn. Thực hiện bước này khi cần cập nhật lên phiên bản mới hơn hoặc sau khi cài lại OS.

Cập nhật qua DGX Dashboard (khuyến nghị)

```
bash

# Truy cập trực tiếp trên màn hình gắn GX10
# Mở trình duyệt và vào:
http://localhost:11000

# Hoặc SSH tunnel từ máy khác:
ssh -L 11000:localhost:11000 user@192.168.100.10
# Mở trình duyệt trên máy cục bộ: http://localhost:11000
```

Vào **System Updates** → cập nhật toàn bộ → reboot nếu có yêu cầu.

⚠ **Không dùng** `sudo apt upgrade` — có thể ghi đè driver Blackwell do NVIDIA quản lý.

Cài Docker & NVIDIA Container Toolkit

Thực hiện trên **cả 2 máy** — GX10 với DGX OS thường đã có Docker; kiểm tra trước khi cài.

```
bash
```

```
docker --version && nvidia-ctk --version

# Nếu chưa có Docker:
curl -fsSL https://get.docker.com | sudo bash
sudo usermod -aG docker $USER
newgrp docker

# Nếu chưa có NVIDIA Container Toolkit:
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Xác nhận GPU hoạt động trong container
docker run --rm --runtime nvidia --gpus all ubuntu nvidia-smi
```

IV. Cài vLLM

GX10 dùng CUDA 13.x — wheel vLLM thông thường từ PyPI biên dịch cho CUDA 12.x sẽ báo lỗi:

```
libcudart.so.12: cannot open shared object file: No such file or directory
```

Phải dùng Docker image có CUDA 13 / aarch64 từ dự án `spark-vllm-docker`.

1. Clone spark-vllm-docker

Thực hiện trên cả 2 máy:

```
bash
```

```
git clone https://github.com/eugr/spark-vllm-docker.git
cd spark-vllm-docker
```

2. Đăng nhập HuggingFace

Cả 3 model là **gated model**, phải accept license trên trang HuggingFace trước khi download:

Model	URL
MedGemma 27B-IT	https://huggingface.co/google/medgemma-27b-it
MedGemma 1.5 4B-IT	https://huggingface.co/google/medgemma-1.5-4b-it
Llama 4 Scout 17B	https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct
Tạo token	https://huggingface.co/settings/tokens (chọn loại Read)

Khai báo token trên **cả 2 máy**:

```
bash

# Đăng nhập interactive
hf auth login

# Hoặc export thủ công (thay hf_xxxx bằng token thực)
echo 'export HF_TOKEN=hf_xxxxxxxxxxxxxxxx' >> ~/.bashrc
source ~/.bashrc
```

Kiểm tra đã đăng nhập thành công:

```
bash
```

```
hf auth whoami
```

```
echo $HF_TOKEN
```

```
# → phải hiện tên tài khoản và token, không phải dòng trống
```

3. Tạo các file cấu hình recipe trong thư mục `recipes/`

`recipes/MedGemma-4b-1.5-it-bf16.yaml`

```
yaml
```

```
recipe_version: "1"
name: MedGemma-1.5-4B-IT
description: vLLM serving MedGemma-1.5-4B-IT (bfloat16)

model: google/medgemma-1.5-4b-it

cluster_only: false
solo_only: false

container: vllm-node-tf5-medgemma4b

build_args:
  - --tf5

mods: []

defaults:
  port: 8002
  host: 0.0.0.0
  gpu_memory_utilization: 0.1
  max_model_len: 8192
  max_num_batched_tokens: 8192
  max_num_seqs: 8

env: {}

command: |
  vllm serve google/medgemma-1.5-4b-it \
    --dtype bfloat16 \
    --quantization fp8 \
    --kv-cache-dtype fp8 \
    --gpu-memory-utilization {gpu_memory_utilization} \
```

```
--max-model-len {max_model_len} \  
--max-num-batched-tokens {max_num_batched_tokens} \  
--max-num-seqs {max_num_seqs} \  
--port {port} \  
--host {host} \  
--load-format fastsafetensors \  
--enable-prefix-caching
```

recipes/MedGemma-27b-it-bf16-ray.yaml

yaml


```
recipe_version: "1"
name: MedGemma-27B-IT
description: vLLM serving MedGemma-27B-IT (bfloat16)
```

```
model: google/medgemma-27b-it
```

```
cluster_only: false
solo_only: false
```

```
container: vllm-node-tf5-medgemma27b
```

```
build_args:
  - --tf5
```

```
mods: []
```

```
defaults:
  port: 8001
  host: 0.0.0.0
  gpu_memory_utilization: 0.36
  max_model_len: 32768
  max_num_batched_tokens: 49152
  max_num_seqs: 2
  tensor_parallel: 1
  pipeline_parallel: 2
```

```
env: {}
```

```
command: |
  vllm serve google/medgemma-27b-it \
    --dtype bfloat16 \
    --quantization fp8 \
```

```
--kv-cache-dtype fp8 \  
--gpu-memory-utilization {gpu_memory_utilization} \  
--max-model-len {max_model_len} \  
--max-num-batched-tokens {max_num_batched_tokens} \  
--max-num-seqs {max_num_seqs} \  
--port {port} \  
--host {host} \  
--load-format fastsafetensors \  
--enable-prefix-caching \  
--enable-chunked-prefill \  
--enable-auto-tool-choice \  
--tool-call-parser pythonic \  
--tensor-parallel-size {tensor_parallel} \  
--pipeline-parallel-size {pipeline_parallel} \  
--distributed-executor-backend ray
```

recipes/Llama-4-Scout-17B-16E-Instruct-fp8-ray.yaml

yaml

```
recipe_version: "1"
name: Llama-4-Scout-17B-16E-Instruct
description: vLLM serving Llama-4-Scout-17B-16E-Instruct-FP8 (dtype auto, tensor-parallel 2 nodes)

model: nvidia/Llama-4-Scout-17B-16E-Instruct-FP8

cluster_only: false
solo_only: false

container: vllm-node-tf5-llama4scout

build_args:
  - --tf5

mods: []

defaults:
  port: 8000
  host: 0.0.0.0
  gpu_memory_utilization: 0.5
  max_model_len: 8192
  max_num_batched_tokens: 4096
  max_num_seqs: 8
  tensor_parallel: 1
  pipeline_parallel: 2

env: {}

command: |
  vllm serve nvidia/Llama-4-Scout-17B-16E-Instruct-FP8 \
    --dtype auto \
    --kv-cache-dtype fp8 \
```

```
--max-model-len {max_model_len} \  
--gpu-memory-utilization {gpu_memory_utilization} \  
--max-num-batched-tokens {max_num_batched_tokens} \  
--max-num-seqs {max_num_seqs} \  
--port {port} \  
--host {host} \  
--tensor-parallel-size {tensor_parallel} \  
--pipeline-parallel-size {pipeline_parallel} \  
--distributed-executor-backend ray
```

V. Chạy các model

Chạy trên **Node 1**. Cờ `-d` chạy container ở chế độ nền (detached).

Bước 1 — Discover cluster (chỉ chạy lần đầu)

Chạy trên **Node 1** để `run-recipe.sh` nhận diện và kết nối các node trong cluster:

```
bash  
  
cd ~/spark-vllm-docker  
./run-recipe.sh --discover
```

Tham khảo: [spark-vllm-docker/recipes](#)

Bước 2 — Start

```
bash
```

```
cd ~/spark-vllm-docker
```

```
./run-recipe.sh MedGemma-4b-1.5-it-bf16 --setup --name=vllm_node_4b -d
```

```
./run-recipe.sh MedGemma-27b-it-bf16-ray --setup --name=vllm_node_27b -d
```

```
./run-recipe.sh Llama-4-Scout-17B-16E-Instruct-fp8-ray --setup --name=vllm_node_llama4scout -d
```

Kiểm tra sau khi các model đã load xong:

```
bash
```

```
curl http://192.168.100.10:8002/v1/models # MedGemma 1.5 4B-IT
```

```
curl http://192.168.100.10:8001/v1/models # MedGemma 27B-IT
```

```
curl http://192.168.100.11:8000/v1/models # Llama 4 Scout 17B
```

Bước 3 — Stop

```
bash
```

```
cd ~/spark-vllm-docker
```

```
./launch-cluster.sh --name vllm_node_llama4scout stop
```

```
./launch-cluster.sh --name vllm_node_27b stop
```

```
./launch-cluster.sh --name vllm_node_4b stop
```

VI. Kiểm tra & giám sát

Kiểm tra tất cả models đã load

```
bash

curl http://192.168.100.10:8001/v1/models # MedGemma 27B-IT
curl http://192.168.100.10:8002/v1/models # MedGemma 1.5 4B-IT
curl http://192.168.100.11:8000/v1/models # Llama 4 Scout 17B
```

Test inference

```
bash
```

```
# Test MedGemma 1.5 4B-IT
curl http://192.168.100.10:8002/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "google/medgemma-1.5-4b-it",
  "messages": [{"role": "user", "content": "Triệu chứng của viêm phổi là gì?"}],
  "max_tokens": 200
}'

# Test MedGemma 27B-IT
curl http://192.168.100.10:8001/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "google/medgemma-27b-it",
  "messages": [{"role": "user", "content": "Phân tích kết quả xét nghiệm máu bình thường."}],
  "max_tokens": 200
}'

# Test Llama 4 Scout 17B
curl http://192.168.100.11:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "nvidia/Llama-4-Scout-17B-16E-Instruct-FP8",
  "messages": [{"role": "user", "content": "Hello, what can you do?"}],
  "max_tokens": 200
}'
```

Theo dõi tài nguyên

```
bash
```

```
# Trạng thái GPU (unified memory – "Not Supported" là bình thường)
nvidia-smi

# RAM hệ thống
free -h

# Xem log container
docker logs -f vllm_node_4b
docker logs -f vllm_node_27b
docker logs -f vllm_node_llama4scout

# Danh sách container đang chạy
docker ps

# DGX Dashboard – giao diện đồ họa đầy đủ (GPU, RAM, Disk, Network)
# http://localhost:11000
```

Khởi động lại sau reboot

Sau khi reboot, thực hiện lại theo thứ tự:

1. Tắt swap: `sudo swapoff -a`
 2. Discover cluster: `./run-recipe.sh --discover` (chỉ cần nếu cấu hình node thay đổi)
 3. Chạy Start (Bước 2 ở trên)
-

VII. Liên hệ hỗ trợ

	Link
ASUS Support GX10	https://www.asus.com/vn/support/
ASUS FAQ GX10	https://www.asus.com/support/faq/1056142/
ASUS GX10 GitHub Discussions	https://github.com/orgs/asus-ascent-gx10/discussions
NVIDIA Developer Forums (GB10)	https://forums.developer.nvidia.com/c/accelerated-computing/dgx-spark-gb10/719
spark-vllm-docker	https://github.com/eugr/spark-vllm-docker
NVIDIA DGX Spark Playbooks	https://github.com/NVIDIA/dgx-spark-playbooks

Phiên bản: 05/2026 · Tài liệu kỹ thuật nội bộ · Dựa trên ASUS Ascent GX10 Official Documentation, NVIDIA DGX Spark User Guide và NVIDIA DGX OS 7 User Guide